

1. Implement K-Means clustering from scratch (centroid initialization, assignment step, update step, convergence check) without using sklearn.cluster.KMeans. Handle the empty-cluster edge case (a centroid that gets zero points assigned).

Code :

```
import numpy as np

# Sample dataset
X = np.array([
    [1, 2], [2, 1], [1, 1],
    [8, 8], [9, 8], [8, 9],
    [5, 6], [6, 5], [5, 5]
])

k = 3
max_iter = 100

# Step 1: Initialize centroids randomly
np.random.seed(42)
indices = np.random.choice(len(X), k, replace=False)
centroids = X[indices]

for _ in range(max_iter):

    # Step 2: Assign each point to nearest centroid
    labels = []
    for point in X:
        distances = [np.linalg.norm(point - c) for c in centroids]
        labels.append(np.argmin(distances))
    labels = np.array(labels)

    # Step 3: Update centroids
    new_centroids = []
    for i in range(k):
        cluster = X[labels == i]

        # Empty cluster handling
        if len(cluster) == 0:
            new_centroids.append(X[np.random.randint(len(X))])
        else:
```

```
new_centroids.append(cluster.mean(axis=0))
```

```
new_centroids = np.array(new_centroids)
```

```
# Step 4: Convergence check
```

```
if np.allclose(centroids, new_centroids):
```

```
break
```

```
centroids = new_centroids
```

```
# Output
```

```
print("Final Centroids:")
```

```
print(centroids)
```

```
print("\nCluster Assignment:")
```

```
for i in range(len(X)):
```

```
print(X[i], "-> Cluster", labels[I])
```

Output :

inal Centroids:

```
[[5.33333333 5.33333333] [1.33333333 1.33333333] [8.33333333  
8.33333333]]
```

Cluster Assignment:

```
[1 2] -> Cluster 1
```

```
[2 1] -> Cluster 1
```

```
[1 1] -> Cluster 1
```

```
[8 8] -> Cluster 2
```

```
[9 8] -> Cluster 2
```

```
[8 9] -> Cluster 2
```

```
[5 6] -> Cluster 0
```

```
[6 5] -> Cluster 0
```

```
[5 5] -> Cluster 0
```



Mevalurkuppam, Tamil Nadu,
India 🇮🇳

22g8+cwh, Mevalurkuppam, Tamil Nadu 602105, India

Lat 13.026432° Long 80.017012°

Wednesday, 26/08/2026 10:15 AM GMT +05:30